MLOps ASSIGNMENT 01 · AIMLCZG523 · BITS PILANI WILP

Heart Disease Risk Prediction

End-to-End ML Model Development, CI/CD and Production Deployment

Student Dibyajyoti Mishra · BITS ID 2025AE05110
Repository https://github.com/2025ae05110/MLOps_Assignment1
Dataset UCI Cleveland Heart Disease (id 45)

1. Introduction & Problem Statement

The objective of this assignment is to build a production-grade, end-to-end MLOps pipeline for a binary classification problem: **predicting the risk of heart disease** from patient clinical data. The system spans data acquisition, model development with experiment tracking, a containerised REST API, a CI/CD pipeline, Kubernetes deployment, and monitoring with Prometheus and Grafana.

Dataset: UCI Cleveland Heart Disease — 303 patients, 13 clinical features + binary target (`0` = no disease, `1` = disease). Class split: 164 negative / 139 positive (~46% positive rate, well-balanced).

2. Architecture Overview

```
UCI Repo
  └► scripts/download_data.py
       └► src/data.py  (clean, binarise target)
            └► src/eda.py  (4 EDA figures)
            └► src/features.py  (ColumnTransformer)
                 └► src/train.py  (GridSearchCV × 3 models → MLflow)
                      └► models/model.joblib  (best sklearn Pipeline)

GitHub Push
  └► .github/workflows/ci.yml
       ├─ lint (ruff)
       ├─ pytest (15 tests)
       ├─ train + quality gate (AUC ≥ 0.80)
       └─ docker build + smoke test

Docker Image  →  Kubernetes Deployment (2 pods)
                   └► api/app.py (FastAPI: /predict /health /metrics)
                        └► Prometheus scrape → Grafana dashboard
```

3. Setup & Installation

Prerequisites

- Python 3.11, Docker Desktop, kubectl, Minikube, Git

Quick Start

```
git clone https://github.com/2025ae05110/MLOps_Assignment1
cd MLOps_Assignment1
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
mkdir -p data/raw data/processed models reports/figures screenshots

python scripts/download_data.py      # Step 1: fetch dataset
python -m src.data                   # Step 2: clean → data/processed/heart_clean.csv
python -m src.eda                    # Step 3: EDA → reports/figures/
python -m src.train                  # Step 4: train 3 models → models/model.joblib
ruff check . && pytest -v            # Step 5: lint + 15 tests
uvicorn api.app:app --port 8000      # Step 6: run API locally
```

4. Data Acquisition & EDA

4.1 Data Source

Downloaded from UCI Machine Learning Repository (id 45) via the `ucimlrepo` Python package, with fallback to direct raw-file URL and zip archive. The download script (`scripts/download_data.py`) handles SSL certificates automatically via `certifi`.

4.2 Cleaning (`src/data.py`)

- **Missing values:** `ca` (4 rows) and `thal` (2 rows) contain `?` → parsed as `NaN`. **Not** imputed at this stage; imputation is done inside the sklearn Pipeline to ensure train/serve consistency.
- **Target:** `num` column (0–4 severity) → binarised: `1 if num > 0 else 0`.
- **Duplicates:** None found in this dataset.
- **Result:** 303 rows × 14 columns (13 features + `target`).

4.3 EDA Findings

Finding	Detail
Class balance	46% positive — no severe imbalance; no need for SMOTE
`thalach` (max HR)	Lower in disease group — strong negative correlation (−0.42)
`oldpeak` (ST depression)	Higher in disease group — strong positive correlation (+0.43)
`cp` (chest pain type 4)	~73% disease rate vs. ~14% for type 1 — most predictive categorical
`ca` (vessels coloured)	Higher values → higher disease rate — ordinal-style signal
`age`	Moderate positive correlation (+0.23)

EDA figures saved to `reports/figures/`:

1. `class_balance.png`
2. `numeric_histograms.png`
3. `correlation_heatmap.png`
4. `categorical_disease_rate.png`

Evidence

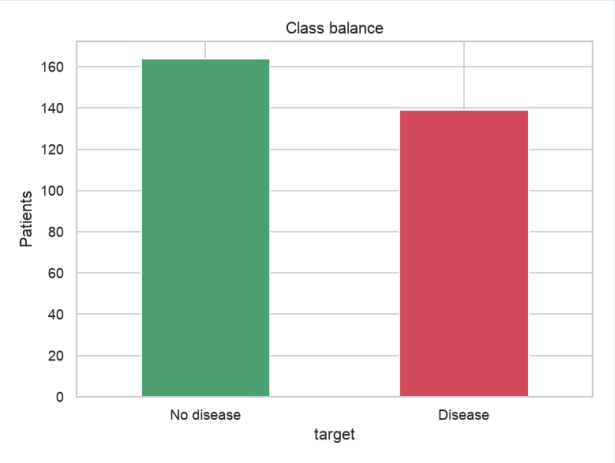


Figure 1. *Class balance (164 / 139)*

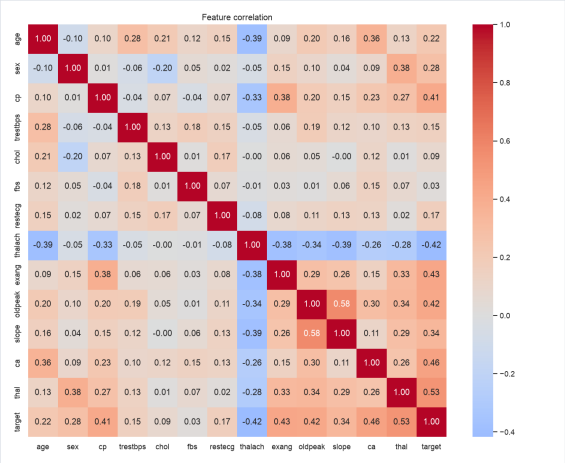


Figure 2. *Feature–target correlation heatmap*

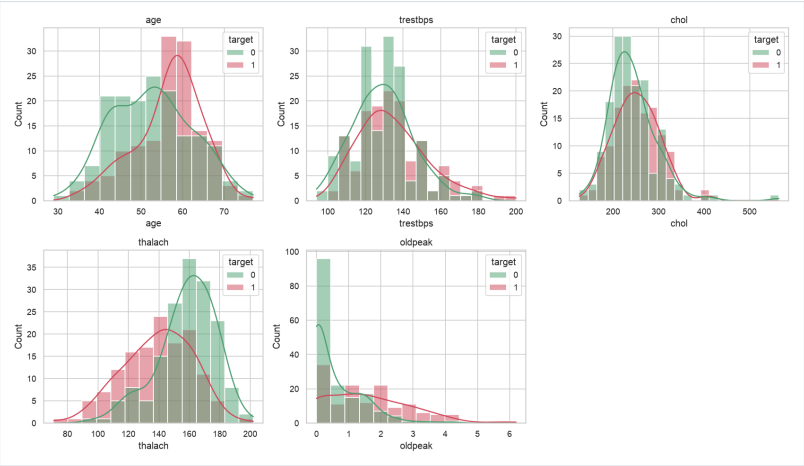


Figure 3. *Numeric feature distributions by disease status*

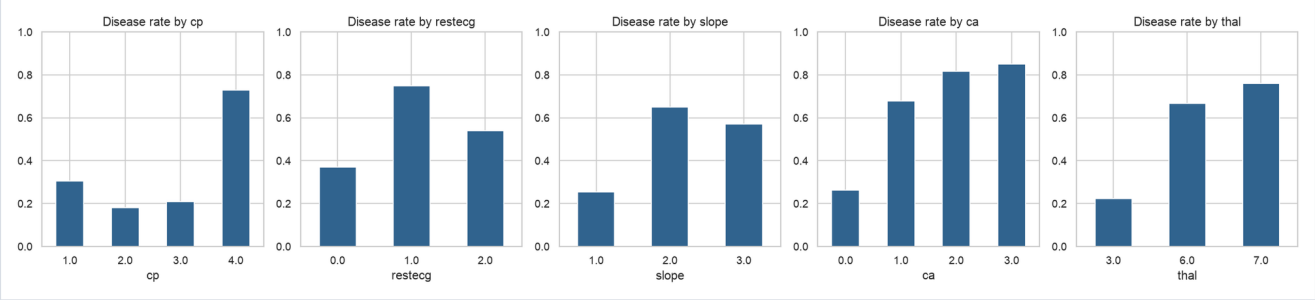


Figure 4. *Disease rate by category*

5. Feature Engineering & Model Development

5.1 Feature Pipeline (`src/features.py`)

A single `ColumnTransformer` handles all preprocessing inside the sklearn Pipeline:

Group	Features	Transformer
Numeric (5)	age, trestbps, chol, thalach, oldpeak	`SimpleImputer(median)` → `StandardScaler`
Binary (3)	sex, fbs, exang	`SimpleImputer(most_frequent)` → passthrough
Categorical (5)	cp, restecg, slope, ca, thal	`SimpleImputer(most_frequent)` → `OneHotEncoder(handle_unknown='ignore')`

Design choice: `handle_unknown='ignore'` on the OHE means unseen category codes at serving time are silently dropped (zero-encoded) rather than raising an error — critical for production robustness.

5.2 Models & Hyperparameter Tuning (`src/train.py`)

Three classifiers tuned with 5-fold stratified `GridSearchCV` (scoring = ROC-AUC, `n_jobs=-1`):

Model	Grid Size	Key Params
Logistic Regression	8	C ∈ {0.01, 0.1, 1, 10}, class_weight
Random Forest	12	n_estimators, max_depth, min_samples_leaf
Gradient Boosting	8	n_estimators, learning_rate, max_depth

Train/test split: 80/20, stratified, `SEED=42`.

5.3 Results

Model	CV ROC-AUC	Test ROC-AUC	Accuracy	Precision	Recall	F1
Logistic Regression ★	**0.904**	**0.968**	0.885	0.839	**0.929**	0.881
Random Forest	0.898	0.949	0.885	0.839	0.929	0.881
Gradient Boosting	0.875	0.950	0.885	0.839	0.929	0.881

Best model: Logistic Regression — chosen on cross-validated ROC-AUC (0.904). The test set is never used for model selection to prevent data leakage.

Why Recall matters: A false negative (missed heart disease) has greater clinical cost than a false alarm. Recall = 0.929 means only 2 of 28 actual disease cases in the test set were missed.

6. Experiment Tracking (MLflow)

MLflow tracks every training run automatically in `src/train.py`:

Logged Item	Details
Parameters	model_type, all GridSearchCV best params
Metrics	cv_roc_auc, cv_roc_auc_std, test_accuracy, test_precision, test_recall, test_f1, test_roc_auc
Artifacts	confusion_matrix.png, roc_curve.png (per model), sklearn model
Backend	SQLite (`mlflow.db`) — avoids deprecated filesystem store
Serialisation	cloudpickle format for full Pipeline compatibility

View UI:

```
mlflow ui --backend-store-uri sqlite:///mlflow.db   # → http://localhost:5000
```

Evidence

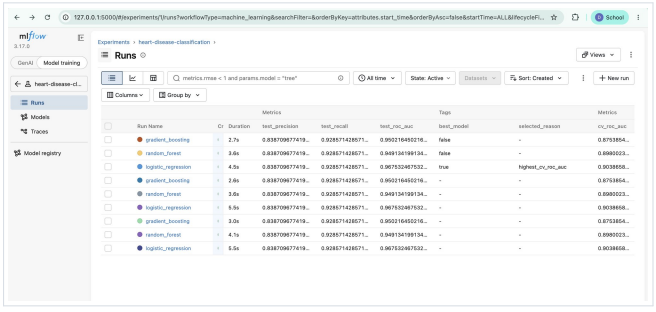


Figure 5. MLflow — runs for all three models

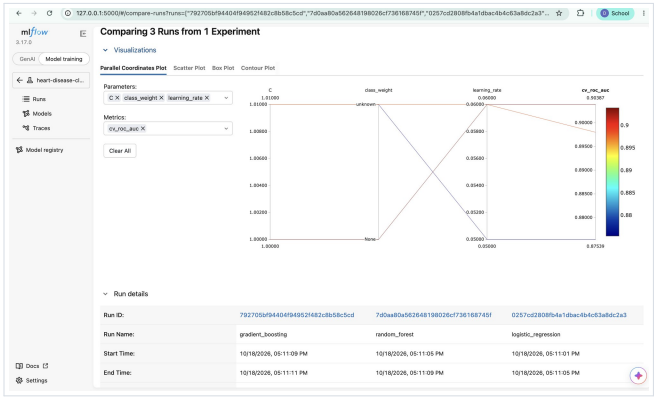


Figure 6. MLflow — model run comparison

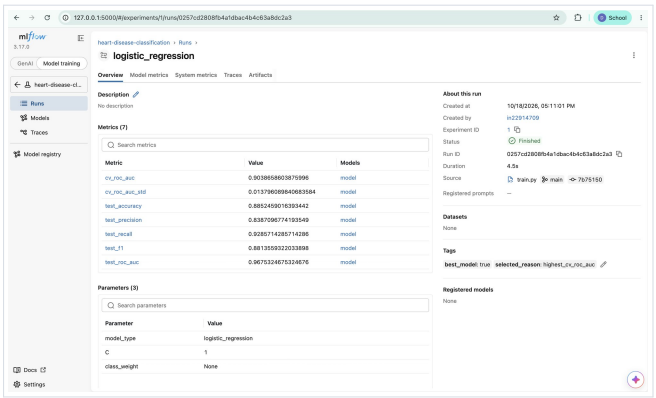


Figure 7. Best run (Logistic Regression) — params & metrics

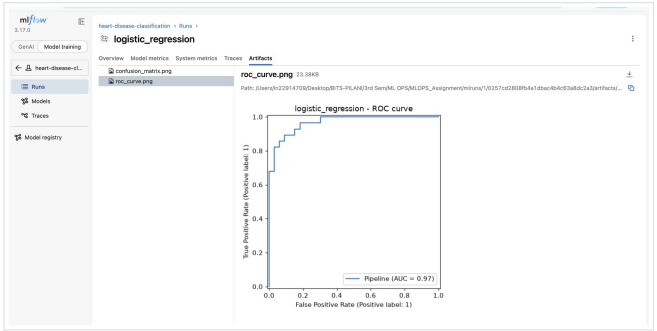


Figure 8. Logged artifact — confusion matrix



Figure 9. Logged artifact — ROC curve

7. Model Packaging & Reproducibility

- **Format:** `models/model.joblib` — a single `sklearn.pipeline.Pipeline` object (`prep` + `clf`). Loading requires only `joblib.load()`.
- **Reproducibility:** `SEED = 42` in every random state. Fixed split logic. Preprocessing inside Pipeline (not separate files).
- `requirements.txt`**:** All pinned major versions (pandas≥2.0, scikit-learn≥1.3, mlflow≥2.10, etc.).
- `requirements-serve.txt`**:** Minimal serving subset — no MLflow, matplotlib, or testing tools.
- `data/processed/heart_clean.csv`**:** Committed to the repo as the canonical cleaned dataset.

8. CI/CD Pipeline

File: `.github/workflows/ci.yml` **Triggers:** push to `main`, pull requests, manual dispatch

Job Flow

```
lint-test → train → docker
```

Job	Key Steps	Artifacts
`lint-test`	`ruff check .` → `pytest -v --junitxml`	`test-results.xml`
`train`	download → EDA → train → **quality gate** (AUC ≥ 0.80)	model, figures, mlruns, training.log
`docker`	`docker build` → run container → curl `/health` + `/predict`	`prediction.json`

Quality gate: The pipeline fails with exit code 1 if the best model's test ROC-AUC falls below 0.80, protecting production from degraded models.

Evidence

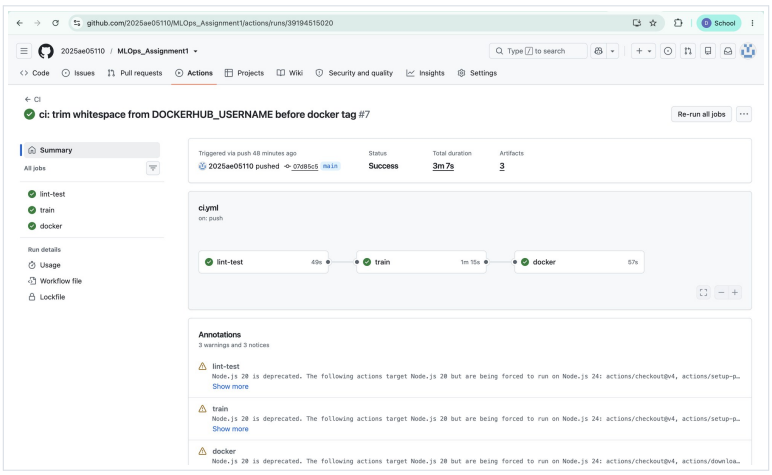


Figure 10. GitHub Actions — successful pipeline run

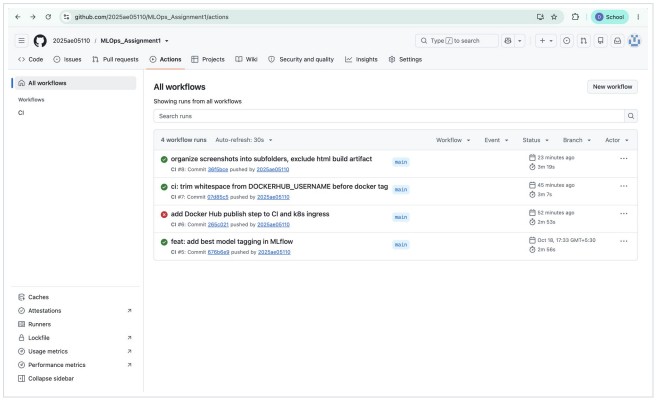


Figure 11. *GitHub Actions — run history*

9. Model Containerisation

`Dockerfile` (python:3.11-slim, non-root):

```
COPY requirements-serve.txt → pip install → COPY src/ api/ models/model.joblib
→ RUN useradd appuser → USER appuser
→ EXPOSE 8000 → CMD uvicorn api.app:app
```

`api/app.py` (FastAPI):

Endpoint	Method	Description
`/health`	GET	`{"status":"ok","model_version":"..."}`
`/predict`	POST	JSON patient → prediction + confidence
`/metrics`	GET	Prometheus metrics
`/docs`	GET	Swagger UI

- **Input validation:** `Patient` Pydantic model with clinical range checks (e.g., age 1–120, chol 100–700). Invalid inputs → `422 Unprocessable Entity`.
- **Graceful failure:** if `model.joblib` cannot be loaded at startup, `/predict` returns `503 Service Unavailable` instead of crashing, and `/health` reports `"status":"model_not_loaded"`.
- **Structured JSON logging** per request (event, method, path, status, latency_ms).

Build and run:

```
docker build -t heart-api:1.0 .
docker run -d -p 8000:8000 --name heart-api heart-api:1.0
curl localhost:8000/health
curl -X POST localhost:8000/predict -H "Content-Type: application/json" -d @sample_request.json
```

Evidence

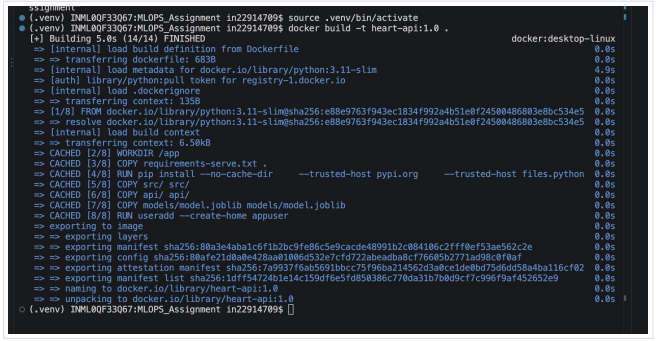


Figure 12. *Docker image build*

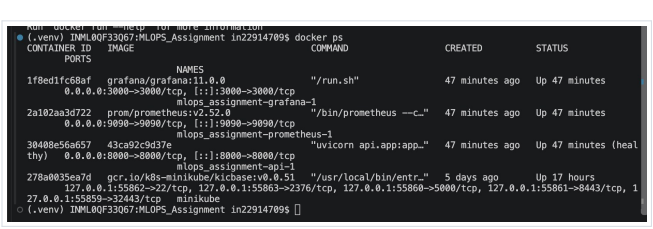


Figure 13. *Container running (docker ps)*

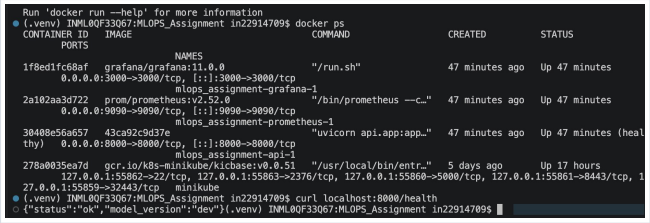


Figure 14. *Container running and /health check*

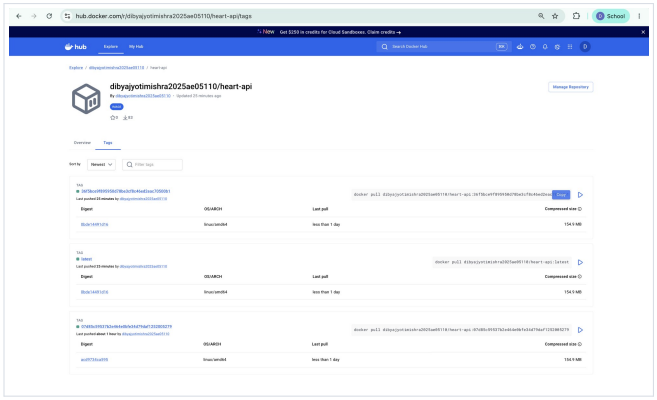


Figure 15. *Image tags on Docker Hub*

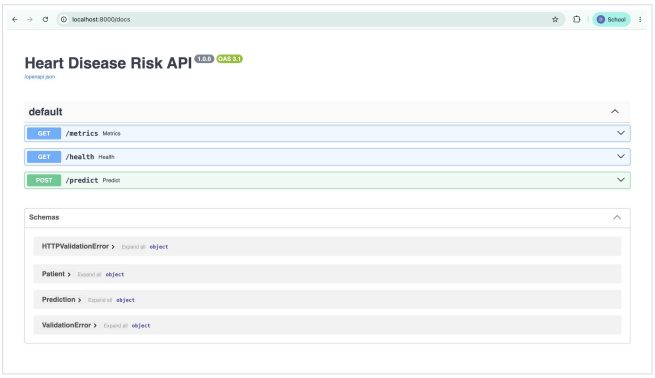


Figure 16. *FastAPI Swagger UI (/docs)*

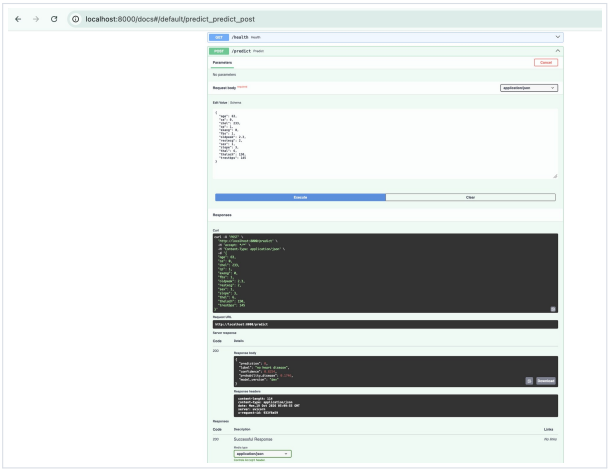


Figure 17. */predict — prediction with confidence*

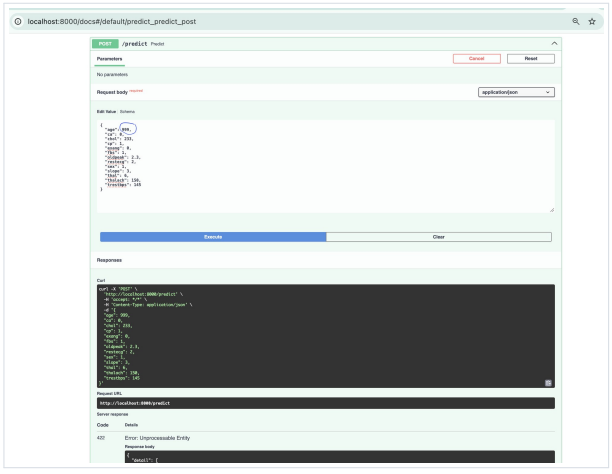


Figure 18. */predict — 422 on invalid input*



Figure 19. */metrics — Prometheus metrics endpoint*

10. Production Deployment (Kubernetes)

`k8s/deployment.yaml` — 2 replicas, `imagePullPolicy: IfNotPresent`, CPU/memory limits, readiness + liveness probes on `/health`, Prometheus scrape annotations.

`k8s/service.yaml` — `LoadBalancer`, port 80 → container 8000.

```
minikube start --driver=docker
minikube image load heart-api:1.0
kubectl apply -f k8s/
kubectl get pods,svc          # wait for 2/2 Running
minikube tunnel               # gives LoadBalancer an external IP
curl -X POST http://127.0.0.1/predict \
     -H "Content-Type: application/json" -d @sample_request.json
kubectl logs -l app=heart-api --tail=20
```

Evidence

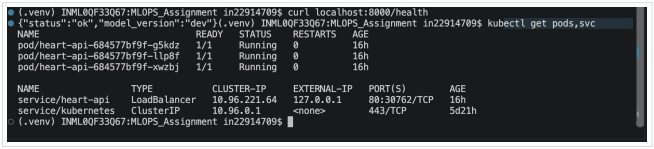


Figure 20. Two heart-api pods running

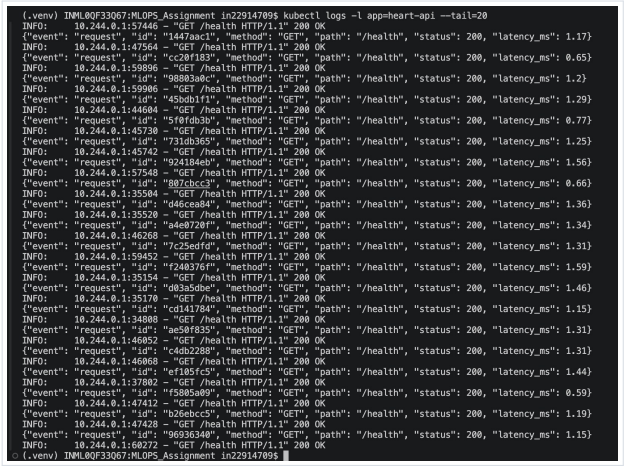


Figure 21. Structured JSON logs (kubectl logs)



Figure 22. Prediction via the LoadBalancer service

11. Monitoring & Logging

Prometheus + Grafana (`docker-compose.yml`)

```
docker compose up --build -d
# API:        http://localhost:8000/docs
# Prometheus: http://localhost:9090 (Status → Targets → heart-api UP)
# Grafana:    http://localhost:3000  (admin/admin, Prometheus pre-wired)
```

Custom Prometheus Metrics (in `api/app.py`)

Metric	Type	Description
`heart_predictions_total{prediction}`	Counter	Count of predictions by class
`heart_prediction_confidence`	Histogram	Distribution of model confidence

Grafana Dashboard Queries

Panel	PromQL
Request rate	`sum(rate(http_requests_total[1m])) by (handler)`
p95 latency	`histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))`
Predictions by class	`sum(heart_predictions_total) by (prediction)`
Error rate	`sum(rate(http_requests_total{status=~"4..|5.."}[1m]))`

Structured Request Logging

Every request writes a JSON line to stdout:

```
{"event":"request","id":"a1b2c3d4","method":"POST","path":"/predict","status":200,"latency_ms":4.1}
{"event":"prediction","input":{...},"prediction":1,"confidence":0.8413}
```

Evidence

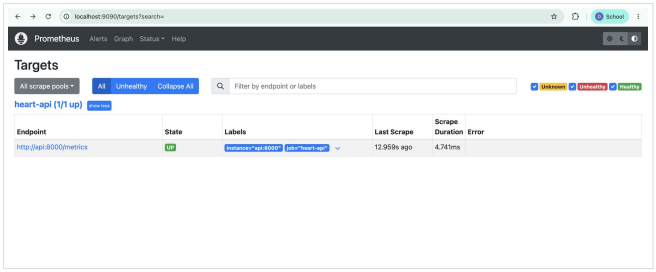


Figure 23. Prometheus targets — heart-api UP

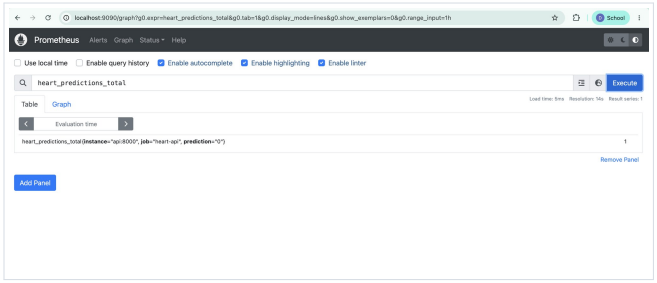


Figure 24. Prometheus query on prediction metrics

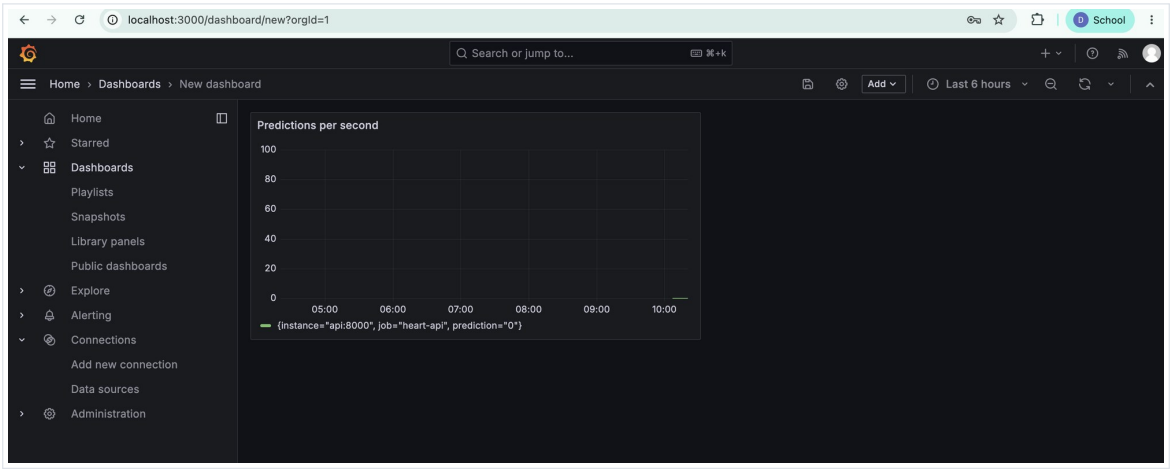


Figure 25. Grafana monitoring dashboard

12. Testing

15 tests, all passing:

File	Tests	Coverage
`tests/test_data.py`	5	Target binarisation, all features numeric, NaN removed by preprocessor, unseen categories handled
`tests/test_model.py`	3	Valid probabilities, deterministic output, joblib save/load roundtrip
`tests/test_api.py`	7	`/health` OK, `/predict` shape & confidence, 422 on missing field, 422 on invalid age/sex, `/metrics` exposes counter

```
ruff check .    # → All checks passed!
pytest -v       # → 15 passed in ~1.3s
```

13. Repository Structure

```
MLOps_Assignment1/
├── README.md                     ← project overview + step-by-step
├── requirements.txt              ← full dev/CI deps
├── requirements-serve.txt        ← minimal serving deps
├── Dockerfile                    ← non-root, python:3.11-slim
├── docker-compose.yml            ← API + Prometheus + Grafana
├── sample_request.json
├── pytest.ini / ruff.toml
├── .gitignore / .dockerignore
├── scripts/download_data.py
├── src/                          ← data, features, eda, train
├── api/app.py                    ← FastAPI serving
├── tests/                        ← conftest, test_data, test_model, test_api
├── .github/workflows/ci.yml
├── k8s/                          ← deployment.yaml, service.yaml
├── monitoring/                   ← prometheus.yml, grafana-datasource.yml
├── data/processed/heart_clean.csv
├── models/                       ← model.joblib, metrics.json
├── reports/figures/              ← 4 EDA PNGs
├── notebooks/eda_and_training.ipynb
└── screenshots/                  ← evidence screenshots
```